

Strategy e State

Padrões de Projeto - Aula #08

Prof. Dr. Carlos Melo

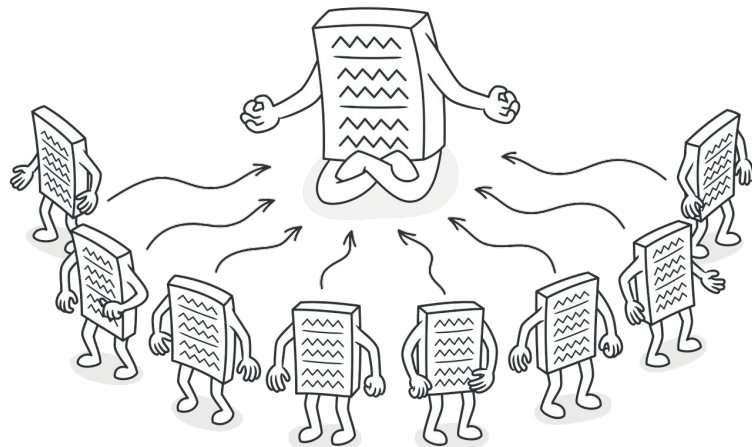
 carlos.melo@upe.br



Padrões Comportamentais (parte #2)

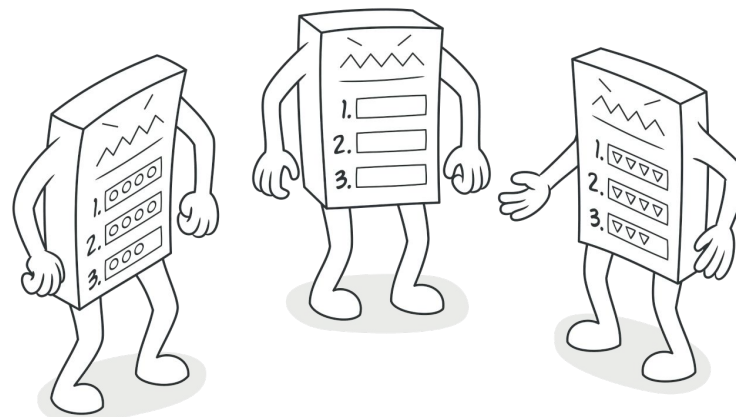
Strategy

Família de algoritmos intercambiáveis, escolhidos em tempo de execução



State

O objeto muda de comportamento quando seu estado interno muda



- Os dois têm diagramas parecidos, o que muda é a intenção.



Objetivos de aprendizagem

- Implementar *strategies* e saber quando uma função basta no lugar de uma classe
- Distinguir Strategy de Template Method e Bridge
- Identificar uma máquina de estados escondida em ifs de algum atributo
- Implementa State e transition (algo que o Strategy não tem)

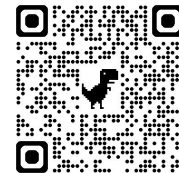


Strategy - O Problema

- Temos o UPEMaps que é um app de navegação.
- A primeira versão só traçava rotas de carro.

```
1 class Navegador:  
2     def construir_rota(self, origem, destino):  
3         # 100+ linhas de algum algoritmo de menor caminho em grafos  
4         return ["seguir BR-232", "converter à direita", "chegou"]
```

- Novos requisitos:
 - rotas a pé
 - rotas de transporte público
 - rotas de bicicleta
 - rotas turísticas passando por pontos de interesse



Strategy - O Problema

```
1 class Navegador:
2     def construir_rota(self, origem, destino, modal):
3         if modal == "carro":
4             ... # 100+ linhas
5         elif modal == "pe":
6             ... # 100+ linhas
7         elif modal == "publico":
8             ... # 100+ linhas
9         elif modal == "bicicleta":
10            ... # 100+ linhas
```

- A classe cresceu 300%
 - Qualquer alteração de algoritmo pode demandar alterar uma classe compartilhada por todos os meios de transporte e ocasionar conflitos na hora de algum merge no github.



Strategy - O Problema

- Usar `if` e `else` não é o problema, mas sim o fato de todos os algoritmos estarem armazenados na mesma classe.
 - Se a classe cresce a cada requisito é sinal de que estamos violando o **O** do **SOLID**, uma vez que cada extensão vai exigir edição.
 - A classe tem quatro responsabilidades, o que nos leva a violar o **S** do **SOLID**.
 - Testar uma rota a pé, requer carregar um objeto `Navegador` com todos os algoritmos.
 - Todo mundo vai editar o mesmo arquivo.
- Além disso, os quatro blocos têm algo em comum:
 - `rota = algoritmo(origem, destino)`
 - Mesma assinatura, mesma finalidade, diferentes implementações.
 - A isso chamamos de família de algoritmos intercambiáveis



Singleton - Intenção

"Garantir que uma classe tenha somente uma instância e fornecer um ponto global de acesso a ela."

- Padrão criacional
 - O menor e mais controverso dos padrões
- Promessas:
 - Uma única instância (o inferno na Terra de testar)
 - Ponto global de acesso (é literalmente uma variável global)
- Torna o construtor inacessível, e um método estático assume o lugar dele (cria na primeira chamada) e depois só retorna a instância caso não exista.



Strategy - Intenção

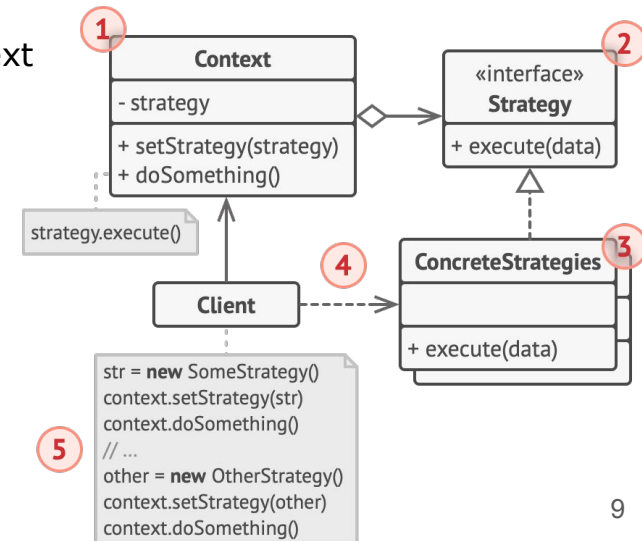
"Definir uma **família** de algoritmos, **encapsular** cada um deles e torná-los **intercambiáveis** .
Strategy permite que o algoritmo varie independentemente dos clientes que o utilizam."

- Padrão comportamental
- Família:
 - Algoritmos que fazem a mesma coisa de formas diferentes
- Encapsular:
 - Cada um em sua própria classe
- Intercambiáveis:
 - O contexto pode ser trocado sem a necessidade de saber qual é qual



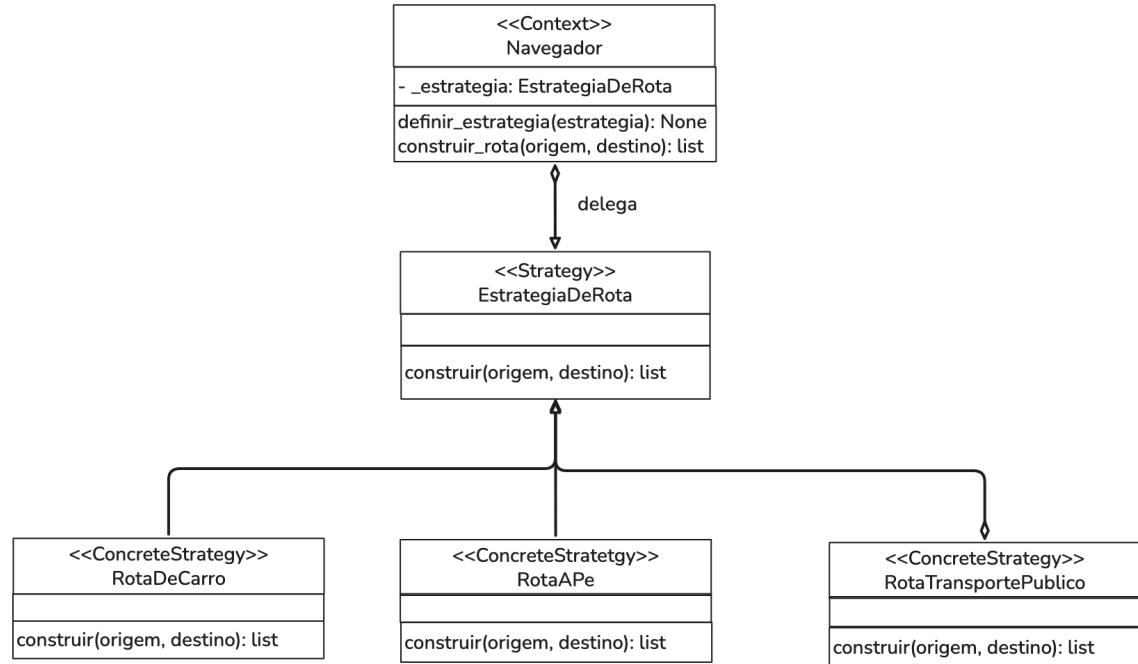
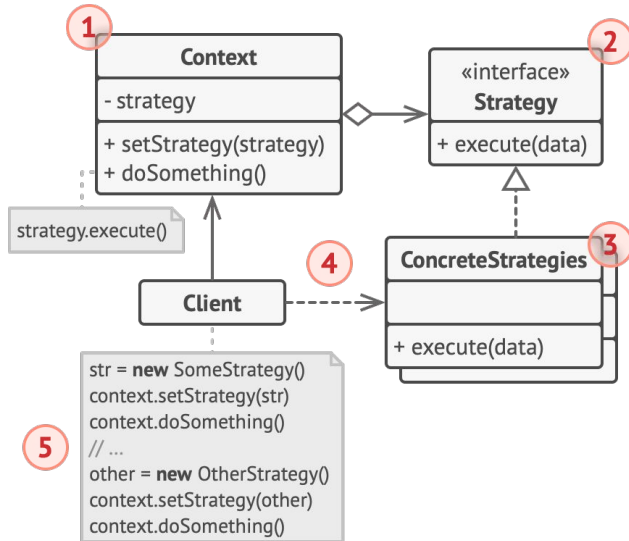
Strategy - Estrutura

1. **Context** mantém uma referência para uma das **ConcreteStrategies** e se comunica com esse objeto através da interface da estratégia
2. A interface da **Strategy** é comum a todas as **ConcreteStrategies**.
3. **ConcreteStrategies** implementam diferentes variações de um algoritmo que **Context** usa.
4. O Context chama o método de execução no objeto strategy ligado cada vez que ele precisa rodar um algoritmo. O contexto não sabe qual tipo de estratégia ele está trabalhando ou como o algoritmo é executado.
5. **Client** cria um objeto Strategy específico e o repassa para o Context





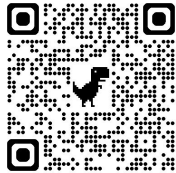
Aplicando ao nosso contexto





Aplicando ao nosso contexto

```
1 from abc import ABC, abstractmethod
2
3 class EstrategiaDeRota(ABC):
4     @abstractmethod
5     def construir(self, origem: str, destino: str) -> list[str]: ...
6
7
8 class RotaDeCarro(EstrategiaDeRota):
9     def construir(self, origem, destino):
10         return [f"Saia de {origem} pela BR-232",
11                "Siga por 230 km",
12                f"Chegou em {destino} - 3h10, 18 L de combustível"]
13
14 class RotaApe(EstrategiaDeRota):
15     def construir(self, origem, destino):
16         return [f"Saia de {origem} pela calçada à direita",
17                "Atravesse a praça central",
18                f"Chegou em {destino} - 47h, evite trechos sem acostamento"]
19
20 class RotaTransportePublico(EstrategiaDeRota):
21     def construir(self, origem, destino):
22         return [f"Terminal de {origem}: ônibus expresso 14:30",
23                "Baldeação em Caruaru",
24                f"Chegou em {destino} - 4h20, R$ 62,00"]
```



Versão Pythonica do Strategy

- A interface tem um método. Em Python é só uma função.

```
1 from typing import Callable
2
3 Estrategia = Callable[[str, str], list[str]]
4
5 def rota_de_carro(origem, destino):
6     return [f"Saia de {origem} pela BR-232", "..."]
7
8 def rota_a_pe(origem, destino):
9     return [f"Saia de {origem} pela calçada à direita", "..."]
10
11
12 class Navegador:
13     def __init__(self, estrategia: Estrategia):
14         self._estrategia = estrategia
15
16     def construir_rota(self, origem, destino):
17         for passo in self._estrategia(origem, destino):
18             print(passo)
```



Versão Pythonica do Strategy

- Use classes quando a estratégia tem estado ou mais de uma operação.
 - Caso contrário, função.
- O partial serve para criar uma nova função fixando antecipadamente alguns valores para os argumentos de uma função já existente.

```
1 Navegador(rota_de_carro).construir_rota("Garanhuns", "Recife")
2
3
4 from functools import partial
5 rota_turistica = partial(rota_de_carro, evitar_pedagio=True)
6 Navegador(rota_turistica).construir_rota("Garanhuns", "Recife")
7
8
9 MODAIS = {"carro": rota_de_carro, "pe": rota_a_pe}
10 Navegador(MODAIS[escolha_do_usuario]).construir_rota(origem, destino)
```



Prós e Contras do Strategy

Prós

- Troca de algoritmo em tempo de execução
- Isola a implementação do algoritmo do código que o usa
- Prioriza composição
- Novas estratégias entram sem tocar no contexto (SOLID)
- Cada algoritmo pode ser testado individualmente sem tocar no contexto

Contras

- Se você tem dois algoritmos que nunca mudam, o padrão é complicação adicional
- O cliente precisa conhecer a estratégia para escolher uma
- Muitas classes pequenas para pouco código

Não use:

- Se tiver um único algoritmo ou quando não precisa realizar a troca em tempo de execução. Muitas vezes o if e else basta.



Template Method - O Problema

- Temos uma aplicação para mineração de dados em documentos corporativos.
- Começamos com arquivos DOC. Depois CSV e PDF.
- Só os dois primeiros passos variam. O processo e a análise são idênticos.
- O cliente se torna apenas um condicional

```
1 if caminho.endswith(".doc"): minerador = MineradorDOC()  
2 elif caminho.endswith(".csv"): minerador = MineradorCSV()  
3 elif caminho.endswith(".pdf"): minerador = MineradorPDF()
```

```
1 class MineradorDOC:  
2     def minerar(self, caminho):  
3         arquivo = self._abrir_doc(caminho)  
4         bruto = self._extrair_doc(arquivo)  
5         dados = self._parsear(bruto)  
6         analise = self._analisar(dados)  
7         self._enviar_relatorio(analise)  
8         arquivo.close()  
9         return analise  
10  
11 class MineradorCSV:  
12     def minerar(self, caminho):  
13         arquivo = self._abrir_csv(caminho)  
14         bruto = self._extrair_csv(arquivo)  
15         dados = self._parsear(bruto)  
16         analise = self._analisar(dados)  
17         self._enviar_relatorio(analise)  
18         arquivo.close()  
19         return analise  
20  
21 class MineradorPDF:  
22     def minerar(self, caminho):  
23         arquivo = self._abrir_pdf(caminho)  
24         ...
```



Template Method - A ideia

- Quebre o algoritmo em passos, escreva a sequência uma vez na classe base, e deixe as subclasses redefinirem só os passos que diferem.

```
1 class MineradorDeDados(ABC):
2     def minerar(self, caminho): # template method
3         arquivo = self._abrir(caminho)
4         bruto = self._extrair(arquivo)
5         dados = self._parsear(bruto)
6         analise = self._analisar(dados)
7         self._enviar_relatorio(analise)
8         self._fechar(arquivo)
9         return analise
```



Template Method - Intenção

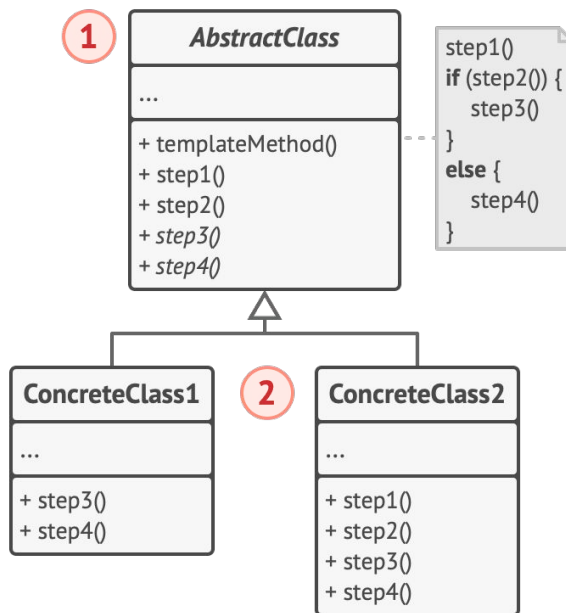
"Definir o **esqueleto** de um algoritmo em uma operação, postergando a definição de alguns passos para subclasses. O Template Method permite que subclasses redefinam certos passos de um algoritmo sem mudar sua estrutura."

- Padrão comportamental
- Esqueleto é a base que controla a ordem
- As subclasses controlam o conteúdo de passos específicos

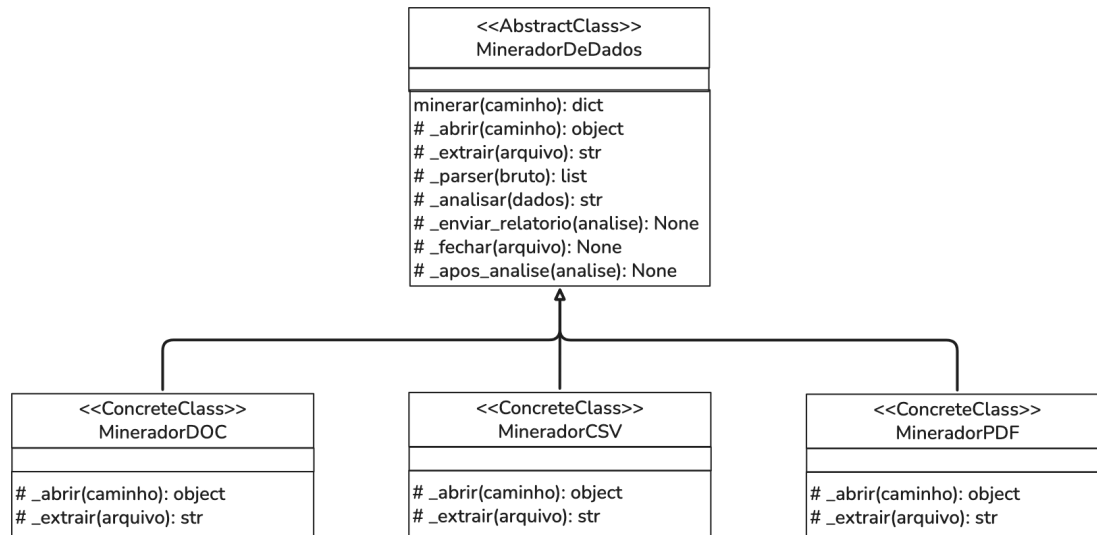
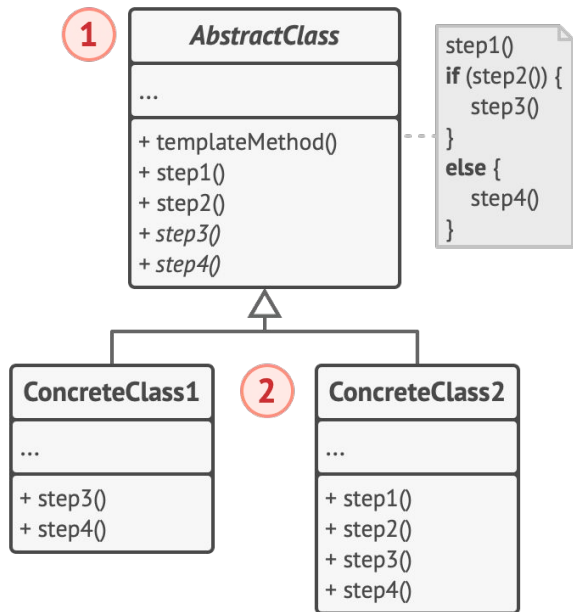
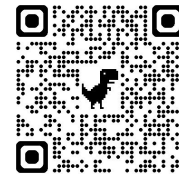


Template Method - Estrutura

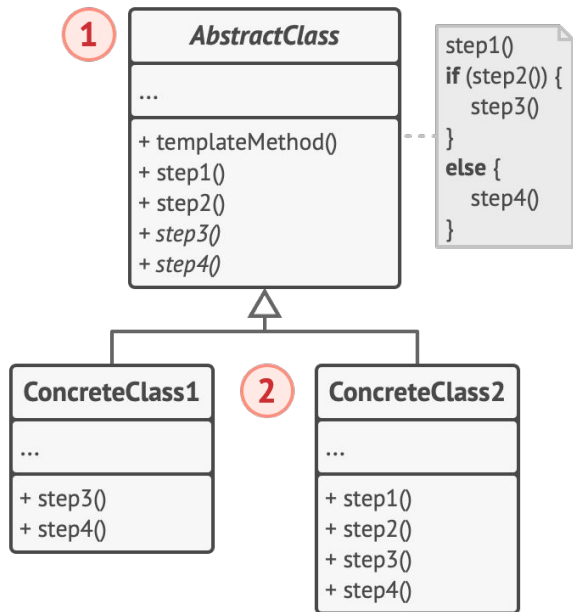
1. **AbstractClass** declara métodos que agem como etapas de um algoritmo, bem como o próprio método padrão (*template*) que chama esses métodos em uma ordem específica.
2. **ConcreteClasses** podem sobrescrever todas as etapas, exceto o método padrão.



Aplicando ao nosso domínio

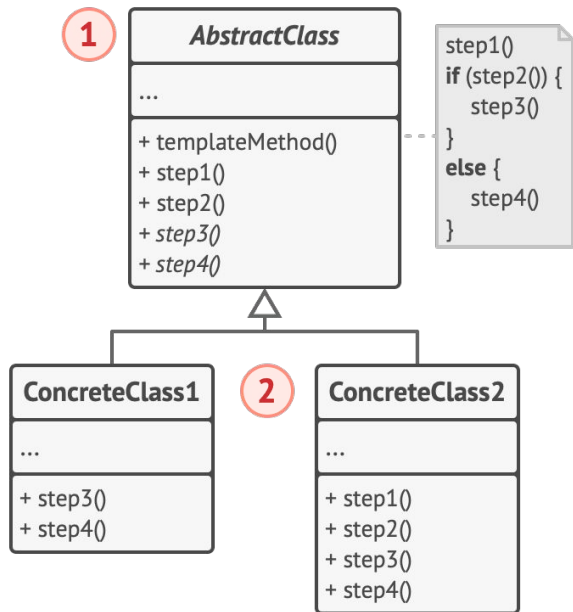
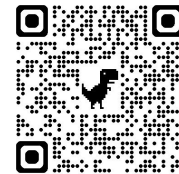


Aplicando ao nosso domínio



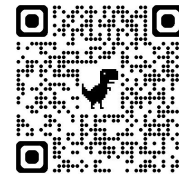
```
1 from abc import ABC, abstractmethod
2
3 class MineradorDeDados(ABC):
4     def minerar(self, caminho: str) -> dict:
5         print(f"Minerando {caminho}")
6         arquivo = self._abrir(caminho)
7         bruto = self._extrair(arquivo)
8         dados = self._parsear(bruto)
9         analise = self._analisar(dados)
10        self._apos_analise(analise)
11        self._enviar_relatorio(analise)
12        self._fechar(arquivo)
13        return analise
14
15 @abstractmethod
16 def _abrir(self, caminho: str): ...
17
18 @abstractmethod
19 def _extrair(self, arquivo) -> str: ...
20
21 def _parsear(self, bruto: str) -> list:
22     return [linha for linha in bruto.split(";") if linha]
23
24 def _analisar(self, dados: list) -> dict:
25     return {"registros": len(dados), "confianca": 1.0}
26
27 def _apos_analise(self, analise: dict) -> None:
28     pass
29
30 def _enviar_relatorio(self, analise: dict) -> None:
31     print(f" [REL] {analise['registros']} registros analisados")
32
33 def _fechar(self, arquivo) -> None:
34     print(" [IO] arquivo fechado\n")
```

Aplicando ao nosso domínio



```
1 class MineradorCSV(MineradorDeDados):
2     def _abrir(self, caminho):
3         print(" [I0] abrindo em modo texto")
4         return f"<csv: {caminho}>"
5     def _extrair(self, arquivo):
6         return "linha1;linha2;linha3"
7
8 class MineradorPDF(MineradorDeDados):
9     def _abrir(self, caminho):
10        print(" [I0] abrindo em modo binário")
11        return f"<pdf: {caminho}>"
12    def _extrair(self, arquivo):
13        print(" [OCR] extraindo texto")
14        return "pagina1;pagina2"
15    def _analisar(self, dados):
16        return {"registros": len(dados), "confianca": 0.87}
17    def _apos_analise(self, analise):
18        print(f" [OCR] confiança média: {analise['confianca']:.0%}")
```

Aplicando ao nosso domínio



```
1 for m in (MineradorCSV(), MineradorPDF()):
2     m.minerar("relatorio_q3")
3
4
5 # Minerando relatorio_q3
6 # [IO] abrindo em modo texto
7 # [REL] 3 registros analisados
8 # [IO] arquivo fechado
9
10 # Minerando relatorio_q3
11 # [IO] abrindo em modo binário
12 # [OCR] extraindo texto
13 # [OCR] confiança média: 87%
14 # [REL] 2 registros analisados
15 # [IO] arquivo fechado
```



Prós e Contras do Template Method

Prós

- Elimina duplicação, uma vez que o algoritmo mora num lugar só
- O cliente pode variar partes de um algoritmo grande.
- Hooks dão pontos de extensão sem exigir que a subclasse saiba da estrutura.
- É a base de alguns frameworks como unittest, Django, Scrapy spiders.
- Pode ser mais rápido que construir, quando há cálculo caro envolvido.

Contras

- Baseado em herança então a subclasse fica presa à classe mãe.
- Viola o **L** do SOLID caso uma subclasse suprima algo da classe mãe de modo incompatível.
- É mais difícil mudar em tempo de execução.

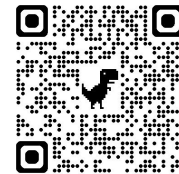
Não use:

- Se os algoritmos forem muito grandes ou se a classe mãe for se tornar muito extensa.



Comparando criacionais

Padrão	Cria a partir de	Mecanismo	Quando
Factory Method	Classe	Herança	Subclasse decide qual produto
Abstract Factory	Classe	Composição	Família que precisa combinar
Builder	Etapas	Composição	Muitos atributos opcionais
Prototype	Outro objeto	Cópia	Copiar é mais fácil que construir
Singleton	“Não cria”	Acesso controlado	Recurso único



Prof. Dr. Carlos Melo

<https://calendly.com/prof-casm>

 carlos.melo@upe.br